

Manejo de Eventos en Vue 3

Interacción, acciones del usuario y respuestas reactivas.



Event Handling Syntax

```
<template>
  <!-- User clicks the button -->
  <button @click="handleClick">
    Click Me
  </button>
</template>

<script>
import { ref } from 'vue';

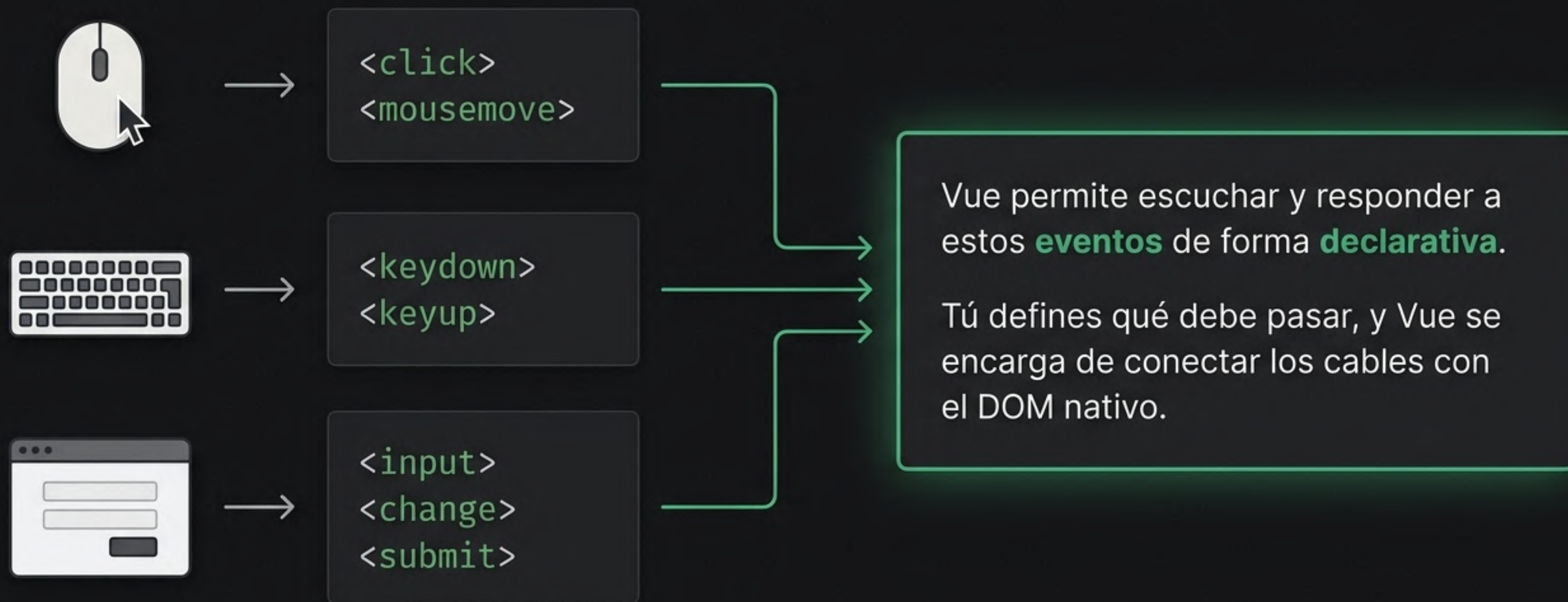
export default {
  setup() {
    // Reactive data
    const count = ref(0);

    // Event handler function
    const handleClick = (event) => {
      // Vue Green highlights the key action
      count.value++;
      console.log('Button clicked!', event);
    };

    return {
      count,
      handleClick
    };
  }
};
</script>
```

El lenguaje del navegador

Un **evento** es cualquier acción que ocurre en la interfaz o el navegador.



Escuchando eventos

La transición de una sintaxis verbosa a una notación elegante.

Sintaxis completa nativa

```
<!-- Sintaxis completa nativa -->  
<button v-on:click="saludar">Saludar</button>
```

Abreviación recomendada

```
<!-- Abreviación elegante (Shorthand) -->  
<button @click="saludar">Saludar</button>
```

La directiva **v-on** se acopla a los **eventos del DOM**. **@** es la abreviación más usada en proyectos Vue modernos.

El Ciclo Reactivo (Tu primer evento)

```
<div id="app">
  <button @click="saludar">{{ mensaje }}</button>
</div>

<script>
  const { createApp, setup, ref } = Vue;
  createApp({
    setup() {
      const mensaje = ref('Haz clic aquí');
      const saludar = () => { mensaje.value = '¡Hola Vue 3!' };
      return { mensaje, saludar };
    }
  }).mount('#app');
</script>
```



¡ATENCIÓN! En Vue, evita manipular el DOM directamente como solución principal. Cambia el estado reactivo y deja que Vue actualice la interfaz.

Ciclo de Reactividad



1. Interacción: El usuario hace @click.



2. Lógica: Se ejecuta la función saludar().



3. Mutación: Cambia el estado reactivo (mensaje.value).



4. Actualización: Vue detecta el cambio y redibuja el DOM automáticamente.

El catálogo de interacciones

Sintaxis Vue	Disparador	Ejemplo de Uso
@click	Clic del ratón	<code><button @click="enviar"></code>
@input	Escritura (pulsación a pulsación)	<code><input @input="buscar"></code>
@change	Cambio confirmado en un input, select o checkbox.	<code><select @change="filtrar"></code>
@submit	Envío de un formulario completo	<code><form @submit="guardar"></code>
@keydown	Tecla presionada	<code><input @keydown="mover"></code>

Manejadores en línea (Inline)

Ejecutan expresiones simples de JavaScript directamente dentro del template.



Buena práctica: Ideales para lógica breve:

```
<button @click="contador++">Sumar</button>  
<button @click="contador = 0">Reiniciar</button>
```



Evitar:

Lógica compleja o múltiples operaciones. Si el código crece y ensucia el template, debe extraerse a una función dedicada.

Lógica estructurada con Funciones

Para lógicas complejas, apuntamos **@evento** a una función definida en nuestro bloque **<script setup>**.

```
<script setup>
import { ref } from 'vue'

const contador = ref(0)

// Manejador declarado como función normal
const incrementar = () => {
  contador.value++
}
</script>

<template>
  <button @click="incrementar">Incrementar</button>
</template>
```

En esta clase usaremos Composition API. Por eso los manejadores serán funciones normales, no la opción methods de Options API.

Dinamismo: Pasando argumentos

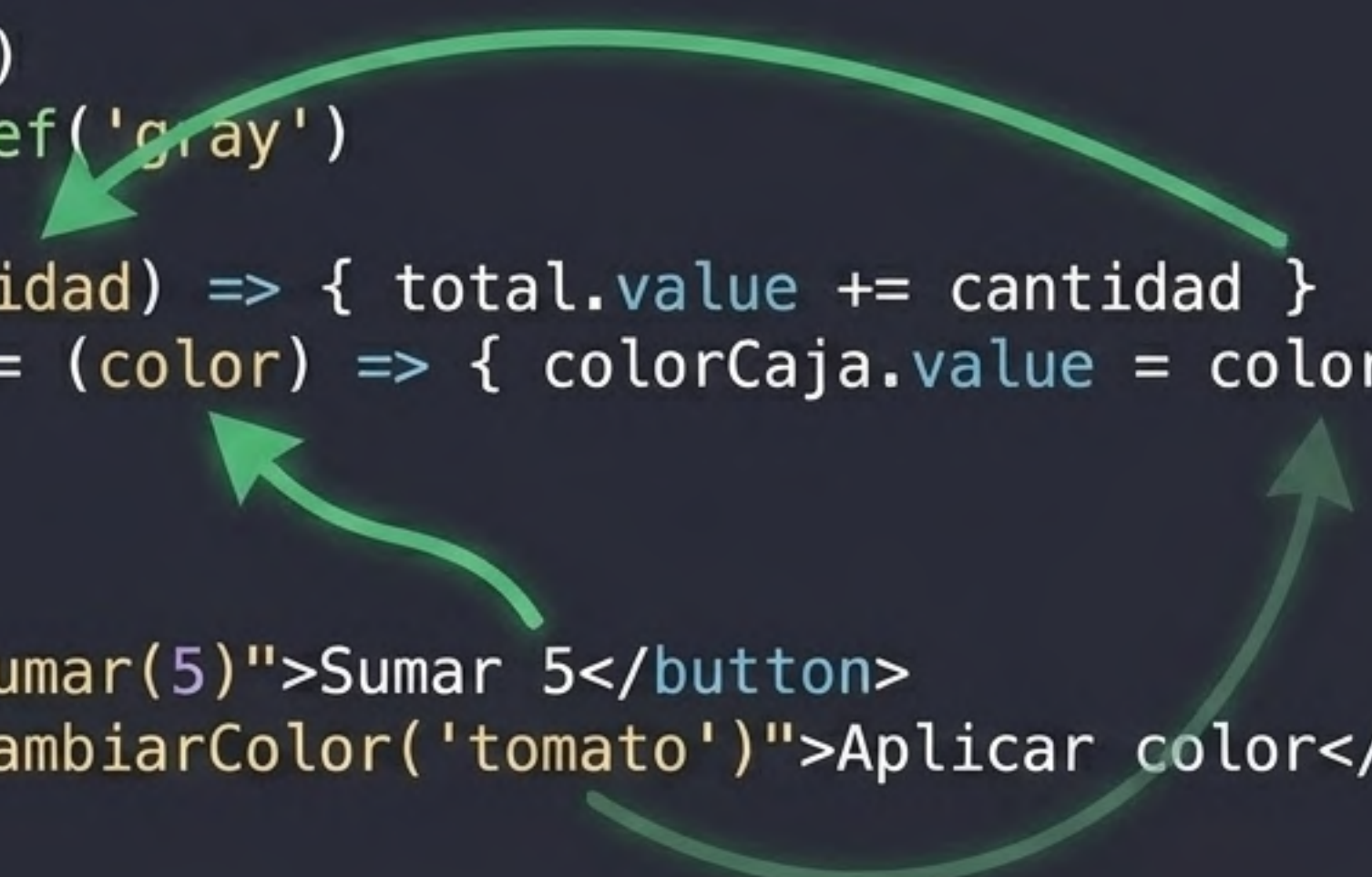
Ejecutar la función en el template nos permite enviar datos personalizados hacia nuestra lógica reactiva.

```
<script setup>
import { ref } from 'vue'

const total = ref(0)
const colorCaja = ref('gray')

const sumar = (cantidad) => { total.value += cantidad }
const cambiarColor = (color) => { colorCaja.value = color }
</script>

<template>
  <button @click="sumar(5)">Sumar 5</button>
  <button @click="cambiarColor('tomato')">Aplicar color</button>
</template>
```

A diagram with two green curved arrows. One arrow starts from the number '5' in the 'sumar(5)' function call within the <button> tag of the template and points to the 'cantidad' parameter in the 'sumar' function definition within the script setup. The second arrow starts from the string 'tomato' in the 'cambiarColor('tomato')' function call within the <button> tag of the template and points to the 'color' parameter in the 'cambiarColor' function definition within the script setup.

Acceso al evento nativo: \$event

Extrae valores o metadatos exactos del elemento del DOM que disparó la acción.

```
<template>
  <input @input="leerTexto($event)">
</template>
```

```
<script setup>
import { ref } from 'vue'
```

```
const texto = ref('')
```

```
const leerTexto = (event) => {
  // Accedemos a la propiedad nativa del input
  texto.value = event.target.value
}
</script>
```

Vue permite usar `$event` para acceder explícitamente al evento original del navegador desde el template.

Modificadores: Menos boilerplate

Sufijos que alteran el comportamiento nativo del evento directamente desde el template, manteniendo tu JavaScript limpio.

```
<form
  @submit
  .prevent="enviar">
  ...
</form>
```

Reemplaza
event.preventDefault().
Impide recargar la
página.

```
<div
  @click
  .stop="accion">
</div>
```

Reemplaza
event.stopPropagation().
Detiene el "burbujeo"
de eventos.

```
<button
  @click.once=
  "aviso">
</button>
```

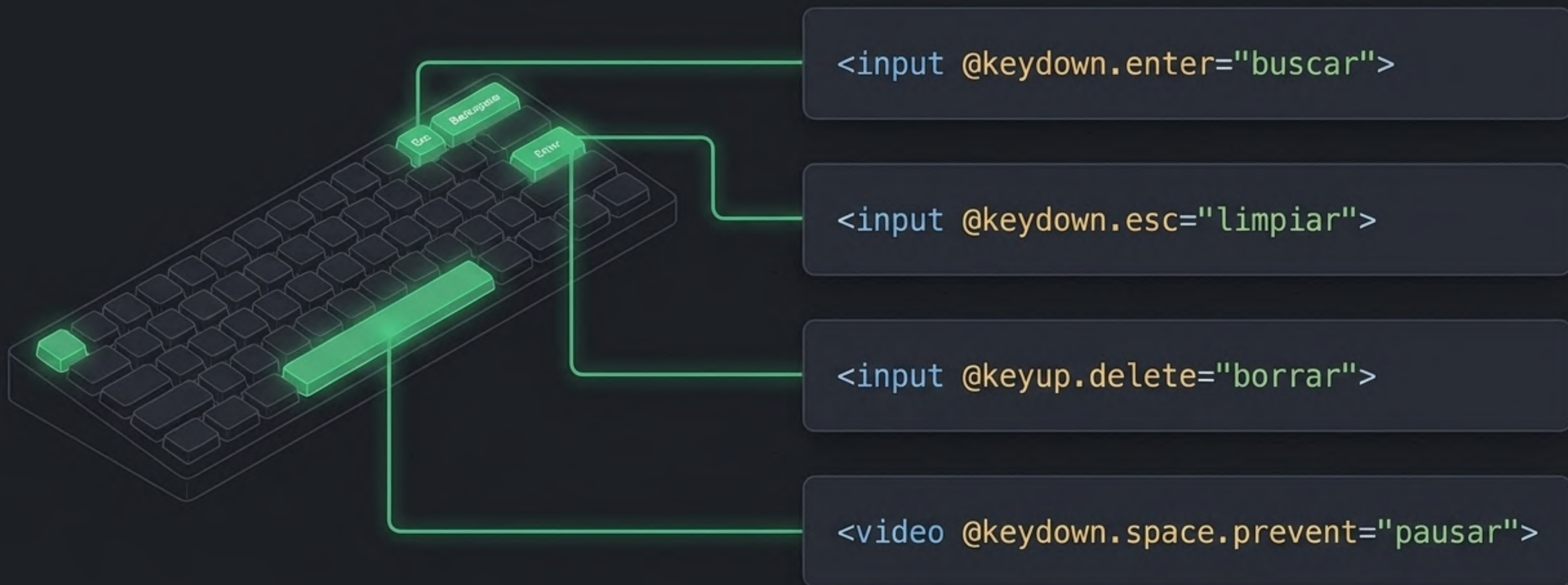
El manejador de
eventos se ejecuta
máximo una sola
vez.

```
<div
  @click.self
  ="cerrar">
</div>
```

Solo se dispara si el
clic ocurre
exactamente en este
elemento (ideal para
fondos oscuros de
modales).

Interacción fluida con el teclado

Ejecuta acciones solo cuando el usuario presiona teclas específicas. Vital para accesibilidad y formularios.



Repaso breve: v-model

Lo usaremos como apoyo para capturar datos desde inputs.

@input

+

:value

=

v-model

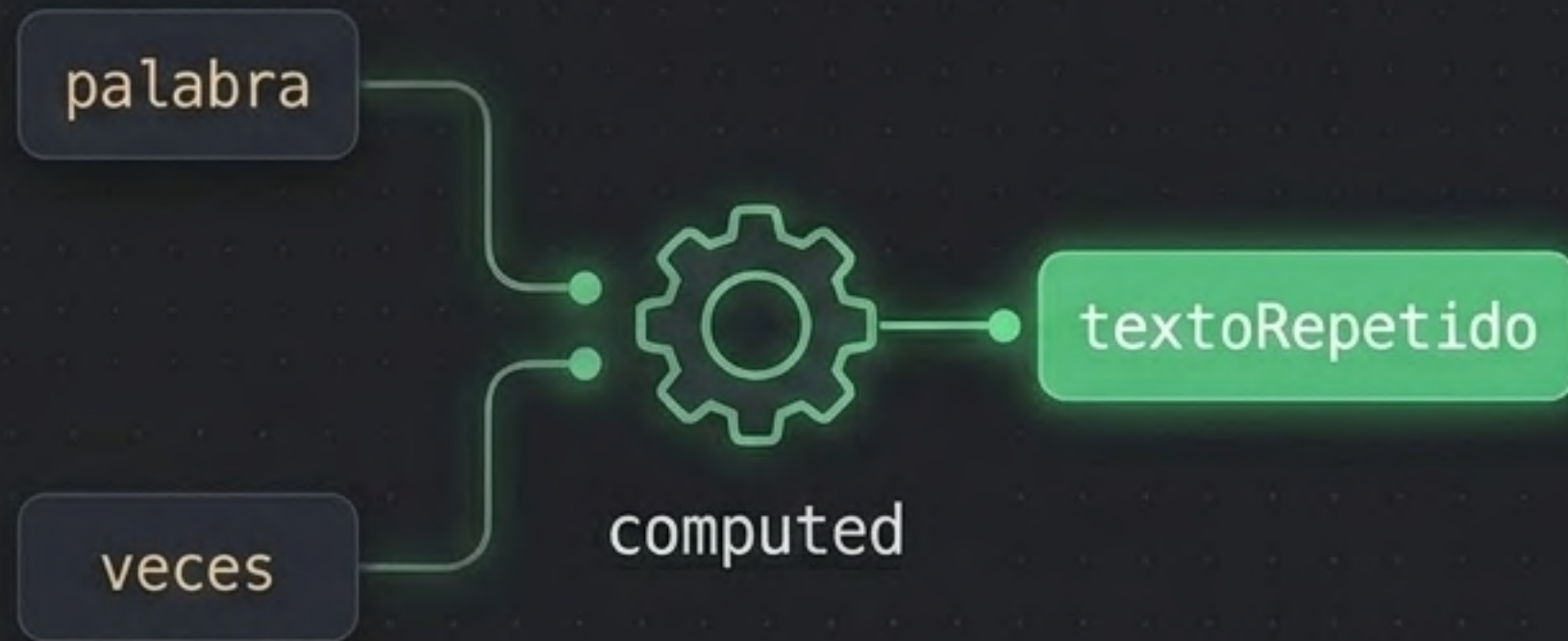
En lugar de capturar manualmente los datos con \$event y reasignarlos al estado, Vue nos regala v-model.

```
<script setup>
import { ref } from 'vue'
const texto = ref('')
</script>

<template>
  <input v-model="texto" placeholder="Escribe algo...">
  <p>Tu texto reactivo: {{ texto }}</p>
</template>
```


Computando respuestas reactivas

El valor derivado se recalcula automáticamente solo cuando sus dependencias cambian.



```
<script setup>
import { ref, computed } from 'vue'

const palabra = ref('Hola')
const veces = ref(3)

// Se actualiza automáticamente si
// 'palabra' o 'veces' cambian
const textoRepetido = computed(() =>
  palabra.value.repeat(veces.value))
</script>
```


Reactividad en acción: mini ejemplos

Repetir texto

```
<script setup>
import { ref, computed } from 'vue'
const palabra = ref('Hola')
const veces = ref(3)

const textoRepetido = computed(() =>
  palabra.value.repeat(veces.value)
)
</script>
<template>
  <p>{{ textoRepetido }}</p>
</template>
```

Calcular

```
<script setup>
import { ref, computed } from 'vue'
const numero1 = ref(5)
const numero2 = ref(10)

const suma = computed(() =>
  numero1.value + numero2.value
)
</script>
<template>
  <p>Suma: {{ suma }}</p>
</template>
```

Invertir texto

```
<script setup>
import { ref, computed } from 'vue'
const texto = ref('Vue.js')

const invertido = computed(() =>
  texto.value.split('').reverse().join('')
)
</script>
<template>
  <p>Invertido: {{ invertido }}</p>
</template>
```

Aplicar color

```
<script setup>
import { ref } from 'vue'
const color = ref('#42b883')
</script>
<template>
  <input type="color" v-model="color">
  <div :style="{ backgroundColor: color }" class="caja">Caja</div>
</template>

<style>
.caja { width: 50px; height: 50px; margin-top: 10px; }
</style>
```


Reglas de Oro en Vue 3

✓ BUENAS PRÁCTICAS

- ✓ Usa @ como abreviación recomendada y muy común para escuchar eventos (@click, @submit).
- ✓ Extrae la lógica compleja a funciones dentro de **<script setup>**.
- ✓ Usa manejadores en línea (@click="contador++") solo para expresiones muy breves.
- ✓ Modifica siempre el estado reactivo (ref), y deja que Vue actualice la interfaz.
- ✓ Aplica modificadores (.prevent, .enter) para mantener tu JavaScript limpio.
- ✓ Apóyate en **v-model** para recolectar datos de inputs fácilmente.
- ✓ Usa **computed** para transformar o derivar cálculos basados en el estado.

✗ ANTI-PATRÓN

- ✗ Evita manipular el DOM directamente. Evita usarlos como solución principal dentro de Vue; piensa primero en estado reactivo.